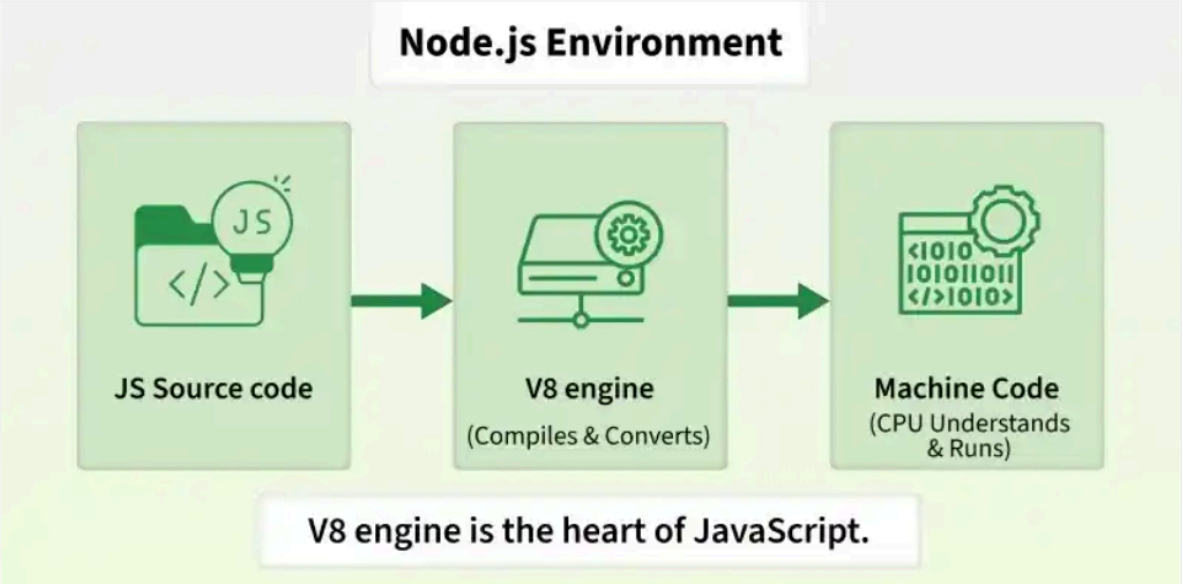
Introduction To Node JS -

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It enables developers to run JavaScript outside the browser to build fast, scalable server-side applications.

- JavaScript was initially a frontend-only language, and Node.js (2009) enabled backend development as well.
- Non-blocking, event-driven architecture for high performance.
- Supports the creation of REST APIs, real-time applications, and microservices.
- Comes with a rich library of modules through npm (Node Package Manager).

*NOTE - Node.js is **not a programming language** and it is **not a framework**. It is a runtime environment for executing JavaScript code without a browser.*



What is the V8 Engine?

The V8 engine is Google's open-source JavaScript engine, used by Chrome and Node.js. Without V8 engine, Node.js would not be able to directly execute JavaScript.

It compiles JavaScript to native machine code for fast execution.

- **Origin:** Developed by Google for Chrome in 2008
- **Integration:** Node.js uses V8 to provide a JavaScript runtime on the server
- **Features:** Just-In-Time compilation, ES6+ support

Creation of Node.js -

In **2009**, **Ryan Dahl** introduced Node.js.

His goal was to create a way to use JavaScript for **server-side programming** with an efficient, event-driven architecture.

Node.js used Google's **V8 JavaScript engine** as its JavaScript execution engine.

Installation of Node.js -

Go to official website - https://nodejs.org/en/download/current
Select the latest version and download the installer according to your operating system. Run the installer and keep the default options.

Verify the installation -

After installation, open:
Windows - Command Prompt / PowerShell on VS code
macOS/Linux - Terminal

Run: node -v

You should get something like:
v26.8.1

The exact version will depend on the current LTS release.

First Node.js program -

Create a folder and open it in VS Code.

Create app.js file.

Write: **console.log("Hello from Node.js!");**

Now open the terminal and run:

node app.js

Output:
Hello from Node.js!

That's your first Node program. Here, you will notice that javascript code ran without any browser because it ran with the help of Node.js. It provided the environment(V8 engine) to run the code.

Node provides its own environment and APIs.

For example, Node can interact with the filesystem:

```
const fs = require("fs");
fs.writeFileSync("hello.txt", "Hello Node!");
```

Run: node app.js

A file called: **hello.txt** will be created.

NOTE - JavaScript is now interacting with the computer rather than just the webpage.

Node.js is widely used for:

- Web servers
- REST APIs
- Real-time applications
- Microservices
- Command-line applications
- Backend applications
- Streaming applications

Features of Node.js -

1. JavaScript Runtime -

Node.js allows us to execute JavaScript outside the browser.

```
const name = "Ekta";
console.log(`Hello ${name}`);
```

Run: **node app.js**

2. Built on V8 Engine -

Node.js uses Google's **V8 JavaScript engine** to execute JavaScript.

V8 converts JavaScript into machine-level instructions efficiently.

3. Asynchronous -

Node.js supports **asynchronous operations**.

For example, when Node.js needs to read a file, it doesn't necessarily have to stop the entire application while waiting for the file operation to finish. The program can continue doing other work while the file operation is being completed.

4. Non-Blocking I/O -

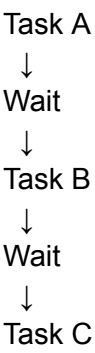
I/O = Input/Output

Examples:

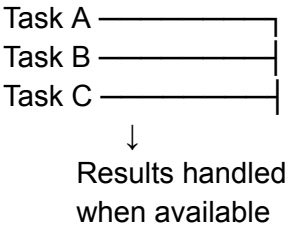
- Reading files
- Writing files
- Database operations
- Network requests

Node.js uses a non-blocking approach for many I/O operations.

Blocking approach



Non-blocking approach



This makes Node.js particularly useful for applications that handle many I/O operations.

5. Event-Driven -

Node.js follows an **event-driven architecture**.

An event can be something like:

- Request received
- File reading completed
- User connected
- Database operation completed

6. Single-Threaded JavaScript Execution -

Node.js executes JavaScript code using a **single main JavaScript thread**. However, this does **not** mean Node.js can only perform one operation at a time. Node.js can handle asynchronous I/O using its event loop and underlying system mechanisms.

7. Fast and Scalable -

Because of its event-driven and non-blocking architecture, Node.js can efficiently handle applications with many concurrent I/O operations.

It is commonly used for:

- Chat applications
- APIs
- Streaming applications
- Real-time applications

8. npm Ecosystem -

npm = Node Package Manager

Node.js has access to a huge ecosystem of reusable packages through **npm (Node Package Manager)**.

It is used to:

- Install packages
- Manage dependencies
- Run scripts
- Manage project configuration

Run on terminal: ***npm -v***

You should also get a version number.

REPL in Node.js -

REPL = Read-Eval-Print Loop

Node.js provides a built-in interactive JavaScript environment called the **Node.js REPL**.
It allows us to execute JavaScript code directly from the terminal without creating a `.js` file.

Starting REPL -

Open the terminal and type:

node

You will see something similar to:

>

Now you can write JavaScript:

> 10 + 20
30

> "Hello".toUpperCase()
'HELLO'

> let x = 100
undefined

> x + 50
150

R — Read
Reads the JavaScript expression entered by the user.

E — Eval
Evaluates/executes the expression.

P — Print
Prints the result.

L — Loop
Waiting for the next command.

REPL is useful for:

- Learning JavaScript
- Testing small pieces of code
- Debugging
- Experimenting with Node.js APIs
- Quickly checking syntax or expressions

Example -

```
> const name = "node"
undefined
> name
'node'
> name.length
4
```

Exit REPL -

You can exit using: **.exit** or press: ***Ctrl + C*** twice.

Global Objects -

Global objects are objects/functions/values that are available throughout a Node.js application without explicitly importing them.

Example 1:

console.log("Hello");

We don't need to import the `console` first.

Example 2:

setTimeout(() => {

console.log("Hello");

}, 2000);

`setTimeout()` can be used directly.

Example 3:

console.log(process.version);

The `process` object provides information and control over the current Node.js process. This displays the Node.js version.